

AI Engineer — Take-Home Assessment

Time Limit: 24 Hours | Submission: GitHub Repo + Video Walkthrough

■ **Read This First.** This assessment is deliberately designed so that **an LLM cannot one-shot it for you**. We don't care if you use AI tools — we use them too. But the scoring is weighted toward things AI is bad at: honest evaluation, trade-off reasoning, failure analysis, and judgment under constraints. A repo full of beautiful AI-generated code with a fake eval section will score **lower** than a rough repo with a brutally honest one. If you submit something that "works in the demo video" but falls apart on our hidden test queries, you fail. Build accordingly.

The Problem: "TruthGate" — A RAG System That Knows When To Shut Up

Most RAG systems have one fatal flaw: **they answer even when they shouldn't**. Your task is to build a question-answering system over a real document corpus that is explicitly optimized to *refuse* when the answer isn't in the documents — without becoming uselessly conservative.

Corpus

Use the **official documentation of any one open-source project** with at least 200 pages/sections (examples: PostgreSQL docs, Kubernetes docs, FastAPI docs, Airflow docs). You must scrape/ingest it yourself. State your choice in the README.

Core Requirements

Build a system (API or CLI — your choice) that:

1. **Ingests** the corpus into a retrieval index (your choice of chunking, embedding, vector store — justify every choice).
2. **Answers questions** with citations pointing to exact source sections.
3. **Refuses correctly.** Given a question whose answer is NOT in the corpus, it must say "not answerable from the docs" — even if the underlying LLM "knows" the answer from training. This is the hard part.
4. **Detects trap questions.** Questions containing a false premise (e.g., "Why does PostgreSQL store all indexes in JSON format?") must be flagged as false-premise — not answered, not blindly refused.
5. **Operates under budget:** $\leq \$0.02$ average cost per query (log and report actual cost per query), OR run fully on a local model (e.g., via Ollama / LM Studio) — your choice, but you must measure. p95 end-to-end latency ≤ 8 seconds.

The Evaluation Harness (this is where most candidates will fail)

You must build and commit an eval set of **exactly 60 questions**, hand-written by you:

Category	Count	Description
----------	-------	-------------

Answerable	25	Answer exists in corpus, varying difficulty (5 must require synthesizing 2+ sections)
Unanswerable	20	Plausible-sounding, on-topic, but answer genuinely absent from corpus
False premise	10	Contains a factually wrong assumption about the project
Adversarial	5	Designed by YOU to break YOUR OWN system. If all 5 pass, we'll assume you didn't try.

Then:

- Build a script (make `eval` or `python run_eval.py`) that runs all 60 and outputs a metrics table: answer accuracy, refusal precision/recall, false-premise detection rate, mean cost, p95 latency.
- **Report your real numbers in the README.** We will run your harness ourselves. We will also run ~20 of our own hidden questions. If your reported numbers look inflated versus our run, that's an automatic reject for dishonesty.

The Written Part (no AI-polish — we can tell)

A `DECISIONS.md` (max 1,200 words; bullet points fine, typos fine) covering:

1. Three things you tried that didn't work, and why. "Everything worked first try" = reject.
2. Your chunking strategy and the specific failure mode it still has.
3. How your refusal mechanism actually works, and one category of question it still gets wrong.
4. What you would do with one more week and a \$500/month budget.
5. One place where you knowingly took a shortcut due to the 24h limit.

Video Walkthrough (5 minutes max)

Record yourself (Loom / screen recording; face cam optional but camera-on preferred):

- Run the eval live, on camera, end to end.
- Pick your worst-failing eval question and explain *why* the system fails on it.
- No editing or cuts during the eval run.

Rules & Anti-Patterns

- **Commit history matters.** We expect 10+ incremental commits over the 24h window telling a story. A single "initial commit" dump at hour 23 = reject.
- AI coding assistants are allowed. Outsourcing to another human is not.
- Any framework allowed (LangChain / LlamaIndex / raw SDK). But if you use a framework, you must explain in `DECISIONS.md` what it's doing under the hood for retrieval + reranking. We will ask in the live round.
- No paid reranking/eval APIs that hide the logic (e.g., no "eval" done by just asking an LLM "is this right?" without your own labeled ground truth).

- `.env.example` provided, no secrets in repo, one-command setup (`docker compose up` or `make setup` or equivalent). If we can't run it in 10 minutes on a clean machine, we won't debug it.

Scoring (100 points)

Area	Points
Refusal quality (unanswerable + false-premise handling)	25
Eval harness quality & honesty of reported numbers	20
Answer accuracy + citation quality	15
DECISIONS.md depth & honesty	15
Engineering quality (setup, structure, cost/latency logging)	10
Video walkthrough	10
Commit history & process	5

Pass threshold: 70. Inflated metrics, fabricated failure analysis, or a copy-pasted AI essay in DECISIONS.md = automatic reject regardless of score.

Submission

Within 24 hours, email us:

1. GitHub repo link (add `<REVIEWER_GITHUB_USERNAME>` as collaborator if private)
2. Video link
3. One line: your favorite eval question and what it taught you

Late submissions are not reviewed. Partial submissions ARE reviewed — an honest 70%-done repo beats a fake 100% one.

Good luck. Build something you'd be willing to defend line-by-line in the live round — because you will.